

How to determine the .NET type to use

The chart below shows how to work out which .NET type to use when declaring an Offset.

Refer to the Documentation

The information about all offsets is given in the FSUIPC documentation:

FSUIPC3: [FSInstallFolder]\Modules\FSUIPC Documents\FSUIPC for Programmers.pdf

FSUIPC4: [FSInstallFolder]\Modules\FSUIPC Documents\FSUIPC4 Offsets Status.pdf

FSUIPC5: [FSInstallFolder]\Modules\FSUIPC Documents\FSUIPC5 Offsets Status.pdf

FSUIPC6: [FSInstallFolder]\Modules\FSUIPC Documents\FSUIPC6 Offsets Status.pdf

FSUIPC7: [YourDocumentsFolder]\FSUIPC7\FSUIPC7 Offsets Status.pdf

Choose a .NET Type

First, find the size of the offset (in Bytes). Then look in the description to see if there is any other description of the type of data stored there. If not then just use the type in the row with the plain number.

e.g. If the offset is 4 bytes use **int** (C#) or **Integer** (VB). However if a 4-byte offset says it's a 32-bit floating point number then use **float** (C#) or **Single** (VB).

Signed or Unsigned?

Some types have two options – a signed version and an unsigned version.

Signed types can store negative and positive values. Unsigned can only contain positive values.

Which type you use depends on the kind of data in the offset. For example:

- **Heading from the compass:** This can only be positive from 0 to 360. So you'll use an Unsigned type.
- **Vertical Speed:** This can be positive or negative (ascending/descending) so you'll use a Signed type.

Types that don't have a signed and unsigned version are always signed.

The Chart:

FSUIPC Offset Size	.NET Framework type	C# Alias	VB.NET Alias	Requires Length?
1	<code>System.Byte</code> (unsigned) <code>System.SByte</code> (signed)	<code>byte</code> <code>sbyte</code>	<code>Byte</code> <code>SByte</code>	No
2	<code>System.Int16</code> (signed) <code>System.UInt16</code> (unsigned)	<code>short</code> <code>ushort</code>	<code>Short</code> <code>UShort</code>	No
4	<code>System.Int32</code> (signed) <code>System.UInt32</code> (unsigned)	<code>int</code> <code>uint</code>	<code>Integer</code> <code>UInteger</code>	No
4 (Specified as 32-Bit Float)	<code>System.Single</code> (signed)	<code>float</code>	<code>Single</code>	No
8	<code>System.Int64</code> (signed) <code>System.UInt64</code> (unsigned)	<code>long</code> <code>ulong</code>	<code>Long</code> <code>ULong</code>	No
8 (Specified as 64-Bit Float or FLOAT64)	<code>System.Double</code> (signed)	<code>double</code>	<code>Double</code>	No
Any specified as string	<code>System.String</code>	<code>string</code>	<code>String</code>	Yes
Any but only useful for offsets storing bit fields.	<code>System.Collections.BitArray</code>			Yes
Any Longitude offset described as being in FS Form or FS Units. (Not decimal degrees)	<code>FsLongitude</code>			Yes (4 or 8)
Any Latitude offset described as being in FS Form or FS Units. (Not decimal degrees)	<code>FsLatitude</code>			Yes (4 or 8)
Any	Array of <code>System.Byte</code>	<code>byte[]</code>	<code>Byte()</code>	Yes